

Module 1

Design & Verification Methodology (Part 1)

Learning objectives

- Spec-to-RTL flow: From SPEC.md (or equivalent) to synthesizable Verilog/SystemVerilog.
- Why verify: The cost of bugs, and how directed tests (and later UVM) reduce risk.
- Minimal toolchain: Verilator + Make + C++ harness for RTL simulation (no UVM in this module).

Prerequisites

- Linux/macOS/WSL2 with a terminal
- Verilator (5.x recommended)
- GNU Make
- C++ compiler (GCC or Clang)

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Specification → RTL Design F"]

T2["Design Methodology"]

T1 --> T2

T3["Intro to Verification: What "]

T2 --> T3

Understand the specification → RTL design flow, design methodology, and why we verify.

Overview

- Module 1 establishes the ****design and verification mindset**** you will use for UART, SPI, and I²C in...

Syllabus topics

1. Specification → RTL Design Flow

- Specification: Document interface (ports, widths, direction), behavior (reset, enable, state transi...
- RTL: Implement the spec in Verilog/SystemVerilog; each requirement in the spec should map to clear...
- Traceability: Being able to point from a line in the spec to the RTL that implements it (and vice v...

2. Design Methodology

- Single block: One spec, one (or a few) RTL files, one testbench.
- Reuse: The same flow (spec → RTL → testbench) scales to protocol blocks (UART, SPI, I²C) in Modules...
- Documentation: Keep SPEC.md (or similar) next to the RTL so that reviewers and verification can che...

3. Intro to Verification: What We Verify, and Why

- What we verify: That the RTL matches the specification (correct function, reset, enable, boundaries...
- Why we verify: Finding bugs in RTL is far cheaper than finding them in silicon or in system integra...
- Directed test: A simple test that drives specific inputs (e.g., reset, then 10 enabled cycles) and...

Command reference highlights

Environment checks

- `verilator --version`

Build and run spec_to_rtl

- `cd module1/examples/spec_to_rtl`

Module script (from repo root)

- `./scripts/module1.sh --check # Environment only`

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --check
```

Terminal: module1_check

```
[[0;36m===== [[0m
[[0;36mModule 1: Demo commands (copy-paste to try) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version

# 2) Run the spec_to_rtl example (spec → RTL → simulation):
cd module1/examples/spec_to_rtl
make run

# 3) Optional: with VCD tracing:
make run TRACE=1
# Then open spec_to_rtl.vcd in a waveform viewer (e.g. gtkwave).

[[1;33m[INFO] [[0m See module1/EXAMPLES.md for more.
```

Example 1: Spec-to-RTL example

- [\[examples/spec_to_rtl/WALKTHROUGH.md\]\(examples/spec_to_rtl/WALKTHROUGH.md\)](#) — Section-by-section rea...

Demo: Spec-to-RTL example

```
$ cd module1/examples/spec_to_rtl && make run
```

Terminal: ex01_spec_to_rtl

```
```bash  
make run
```
```

Expected: `[PASS] spec\_to\_rtl: counter behavior matches spec.`

With VCD tracing (optional):

Practice & assessment

Exercises

- Read the spec and RTL
- Run and extend the test
- Optional: Trace

Assessment checklist

- Can explain the spec → RTL flow and the role of a written specification.
- Can explain what we verify (RTL vs spec) and why verification is done.
- Can run the spec_to_rtl example and describe what SPEC.md, counter.v, and sim_main.cpp do.
- Ready to move to Module 2 (basic TB, UVM+SV, toolchain).

Summary & next steps

- Understand the specification → RTL design flow, design methodology, and why we verify.
- Complete module1/CHECKLIST.md
- Run `./scripts/module1.sh --check`
- Next: Design & Verification Methodology (Part 2)